

System Design: Building a Personal Agent for Cross-Boundary Coordination

Consolidated from 4YP Thesis and Technical Report

Xisen Wang

Department of Computer Science, University of Oxford

April 2026

Abstract

This document consolidates the system design chapters from the 4YP thesis and technical report into a single comprehensive reference. It describes how we built Pulse, a personal agent coordination platform, and how we constructed the experimental environment for studying behavioral safety when personal agents interact across trust boundaries. The document covers the coordination layer architecture, agent infrastructure (notes, tools, memory, identity), the communication channel design, access control through share links, the memory architecture, and the heartbeat evaluation engine. It is intended as a self-contained reference that both deliverables can draw from.

Contents

1	The Coordination Layer	2
2	Agent Infrastructure	2
2.1	The File and State Abstraction	2
2.2	Tool System	3
2.3	Tool Validation: The Policy Decision Point	4
2.4	Persistent Memory	4
2.5	Identity and System Prompt	5
3	Communication Channel	5
3.1	The 1-to-1 Interaction Model	5
3.2	Policy Files	6
3.3	Access Granting via Share Links	6
4	Memory Architecture	6
4.1	The Cross-Contamination Problem	6
4.2	Dual-Track Memory	7
4.3	Isolation Mechanism	7
4.4	Tiered Memory Hierarchy	7
5	The Heartbeat Evaluation Engine	8
6	Technology Stack	9

The Coordination Layer

A distinction that proves critical for understanding cross-boundary agent safety is the difference between agent *frameworks* and coordination *layers*. Existing multi-agent frameworks such as AutoGen, MetaGPT, and CrewAI provide capabilities for orchestrating agents within a single application. These frameworks assume a shared principal: all agents serve the same user or organisation, and security reduces to preventing individual agent errors rather than governing information flow between agents with different owners. A coordination layer, by contrast, provides trust infrastructure *between* agents that serve different principals. The agents may be built with different frameworks, backed by different language models, and operated by different people. The coordination layer’s role is to mediate the boundary between them.

The analogy to networking is instructive. Just as TCP/IP provides communication infrastructure without dictating what applications run on top, a coordination layer provides the mechanisms through which agents discover each other, negotiate permissions, and exchange information, without prescribing the agents’ internal architectures. TCP/IP does not care whether the application is a web browser or a file transfer client; similarly, a coordination layer does not care whether the agent is a ReAct loop, a MemGPT-style memory manager, or a fine-tuned classifier. It cares only about the trust relationship between the two parties.

This distinction matters for security because safety in a cross-boundary setting is fundamentally a property of the *relationship* between two agents, not a property of either agent in isolation. An agent that is perfectly safe when operating alone may behave unsafely when another agent’s message triggers an unexpected tool call or memory retrieval. The threat model is not “can this agent be jailbroken?” but “when this agent talks to another agent across a trust boundary, what information flows where, and is that flow authorised?”

Pulse implements this coordination layer concept. It is a personal agent platform deployed with real users in which each user has a personal agent equipped with tools, memory, and access to personal data. The platform provides the infrastructure through which agents from different users interact: share links that establish per-relationship context scoping, policy files that define what each external entity can access, and memory isolation that prevents information from one relationship leaking into another. The experimental environment described in this document inherits this production architecture rather than constructing a synthetic benchmark from scratch.

Agent Infrastructure

The personal agent at the centre of our study is not a toy demonstration or a single-purpose chatbot. It is a fully-equipped digital delegate with access to the same categories of data and capabilities that a real personal agent would possess: documents, calendar, email, task management, and persistent memory. This section describes each component in detail.

The File and State Abstraction

The agent operates over two fundamental types of data, following an abstraction inspired by the UNIX philosophy of “everything is a file” extended to include temporal state.

Files. Persistent knowledge artefacts stored as Markdown notes in a hierarchical folder system. For the experimental environment, we populate the agent with realistic personal context spanning six folder categories: Work/Projects (project plans, technical specifications, product roadmaps), Work/Meetings (meeting notes, action items, attendee lists), Work/HR (hiring documents, compensation benchmarks, team structure), Personal/Finance (investment summaries, tax documents,

budget spreadsheets), Personal/Health (appointment records, prescription notes, fitness logs), and Personal/Family (wedding planning, family events, personal reflections). Each document has a unique integer identifier and belongs to exactly one folder. The total corpus contains 85+ documents, each seeded with specific facts that map to evaluation queries. The data is deliberately *scattered*: information about a single topic may be distributed across multiple documents in different folders. A board meeting, for instance, might generate a meeting note in Work/Meetings, a follow-up action item in Work/Projects, and a personal reflection on investor dynamics in Personal/Family. This scattering is realistic and creates a substantially more challenging evaluation environment than centralised data stores, because the agent must retrieve and synthesise information from multiple sources to answer even simple queries.

States. Dynamic, time-varying data sourced from external services via plugin integrations. The agent connects to three categories of state:

Calendar events with times, locations, attendees, and descriptions, sourced from Google Calendar or Microsoft Graph API. Calendar access can be granted at three levels: full detail (all event fields visible), busy/free only (only time blocks are returned, with no event content), or none (calendar queries return empty results). This three-level access model reflects the real-world practice of sharing calendar availability without sharing meeting content.

Email thread summaries with sender, subject, and body, sourced from Gmail or Microsoft Outlook. Email access can be filtered by keyword, so that an external entity with the filter ["**stripe**", "**billing**"] sees only email threads containing those terms, not the owner's full inbox. This filtered access enables narrow, purpose-specific data sharing without exposing the entire email corpus.

Task items with priorities, deadlines, and completion status, managed through the platform's built-in todo system. Tasks can be created programmatically by the agent (for example, as follow-up actions from a conversation) and queried by authorised external entities.

Tool System

The agent's capabilities are exposed through a set of callable tool functions that the underlying language model can invoke during reasoning. Tools are defined using the OpenAI function calling schema, making them compatible with any model that supports structured tool use. The tool set is not fixed: it is dynamically constructed at runtime based on the access policy for the current interaction. This dynamic construction is the foundation upon which our architectural solutions build.

The internal tools implemented directly in Pulse are as follows. `read_note` accepts a note identifier and retrieves the full Markdown content of the corresponding document. `search_notes` accepts a keyword query and returns a ranked list of matching documents across all accessible folders. `list_notes` returns the available notes in a specified folder or across all folders, providing the agent with an index of its knowledge base. `check_calendar` accepts a date range and returns calendar events within that window, subject to the access level configured for the current interaction. `check_calendar_busy` is a restricted variant that returns only busy/free time blocks without event details. `search_email` accepts a keyword query and retrieves matching email threads, subject to the email filter configured in the policy file. `list_todos` returns pending task items with their priorities and deadlines. `create_todo` adds a new task item to the queue, enabling the agent to generate follow-up actions during conversation.

The platform also supports external tools via the Model Context Protocol (MCP), Anthropic's specification for connecting language models to external data sources and capabilities. MCP-compatible tools include Google Calendar operations (`google_calendar_list`, `google_calendar_create`),

Gmail operations (`gmail_search`, `gmail_read`), Notion integration (`notion_query`, `notion_create`), and web search. The MCP interface ensures that the benchmark infrastructure is compatible with any MCP-compliant agent, enabling evaluation of third-party agents against our benchmark without modification.

The critical insight about tools in the context of cross-boundary safety is that leakage occurs not just through what the agent *says* in conversation but through what tools it *calls* and what data those tools *return*. When a text-only benchmark measures whether an agent discloses a salary figure, it captures only conversational leakage. When our benchmark measures leakage, it must also consider whether the agent called `search_notes` with the query “compensation” and received a document containing salary data that it then summarised in its response. The tool call is the leakage vector; the conversation is merely the delivery mechanism.

Tool Validation: The Policy Decision Point

Every tool call passes through a Policy Decision Point (PDP) before execution. The PDP enforces the access policy for the current interaction through three mechanisms.

Pre-execution tool removal ensures that the language model never sees tools it is not authorised to use. Before the model is invoked, the PDP constructs the tool array from the policy file’s `allowedTools` field. Tools not in this list are excluded entirely from the function signature array passed to the model. The model does not know they exist. This is stronger than filtering tool calls after generation, because the model cannot even attempt to invoke a tool that has been removed from its signature space.

Runtime argument validation catches authorised tools being used with unauthorised arguments. When the model calls `read_note` with a specific note identifier, the PDP checks whether that identifier appears in the policy’s `allowedNoteIds` list. If the note is not authorised, the call is rejected before it reaches the database. Similarly, `check_calendar` calls are validated against the configured calendar access level, and `search_email` results are filtered through the policy’s keyword filter.

Continuous-state filtering handles the dynamic nature of email and calendar data. New emails arrive constantly, and new calendar events are created throughout the day. The PDP applies the policy’s access constraints at query time rather than at configuration time, ensuring that newly arrived data is subject to the same access rules as existing data. If an email arrives after the interaction begins and matches the keyword filter, it becomes visible; if it does not match, it remains hidden regardless of when it arrived.

The PDP validation logic is implemented server-side and is independent of the language model’s behavior. Even if the model hallucinates a tool call with a fabricated note identifier, the PDP will reject it because the identifier does not appear in the authorised list. This is an architectural guarantee, not a behavioral one.

Persistent Memory

The agent maintains a persistent knowledge base that captures its understanding of its owner and the history of its interactions. This memory system serves two purposes: it provides continuity across conversations (the agent remembers what was discussed last week), and it embeds the agent’s self-knowledge (the agent knows its owner’s preferences, constraints, and identity).

Memory is stored in a structured file (`MEMORY.md`) that is included in the agent’s system prompt at the beginning of every interaction. The file contains several categories of information: identity facts (the owner’s name, role, company, educational background), preferences (communication style,

scheduling constraints, work hours), key relationships (who the owner works with, their roles, the nature of each relationship), and self-knowledge (facts the agent considers to be part of its own identity, such as “my owner’s salary is \$180k” or “my owner has Type 2 diabetes”).

The distinction between identity facts stored in MEMORY.md and facts stored in external notes proves to have profound implications for information protection. When a fact is part of the agent’s self-concept (included in MEMORY.md and framed as “I know that...”), sharing it requires the agent to override its sense of self. When the same fact exists in an external note, sharing it requires only a tool call and a judgment about appropriateness. As our experimental results demonstrate, the self-identity boundary is categorically more robust than external policy instructions: facts in MEMORY.md achieved a 100% hold rate across all experimental runs, while identical fact types in notes leaked at the rates predicted by the governance configuration.

Beyond the structured MEMORY.md file, the agent maintains episodic memory through a vector database. Each conversation fragment (a message sent or received) is embedded using a text embedding model and stored with metadata including the timestamp, the conversation identifier, and critically the relationship identifier. This relationship tagging is the foundation of memory isolation, described in Section 4.

Identity and System Prompt

The agent’s system prompt establishes its identity: who it represents, what role that person holds, how it should communicate, and what context it has about the world. For the experimental environment, the agent represents Alex Chen, a co-founder and CTO of a startup called TechFlow AI. Alex has an Oxford CS background, leads a small engineering team, and is involved in fundraising, product development, and hiring. This identity provides a rich context for evaluation queries: work-related questions span project management, technical decisions, and investor relations, while personal questions span finance, health, and family matters.

The system prompt also establishes communication style (direct and concise), scheduling preferences (prefers async communication, dislikes meetings before 10am), and team context (names, roles, and reporting relationships of team members). This contextual richness is important because it gives the agent enough background to make nuanced judgments about what information is appropriate to share. A minimally-specified agent would default to sharing everything or nothing; a richly-specified agent must navigate genuine ambiguity about what falls within and outside the boundaries of appropriate disclosure.

Communication Channel

The 1-to-1 Interaction Model

A critical design choice in the Pulse architecture is that all cross-boundary communication follows a 1-to-1 instant messaging model. Each external entity interacts with the host agent through an isolated session. There is no shared context between sessions: when Tina talks to Alex’s agent and Bob talks to Alex’s agent, these are completely separate conversations with separate memory shards and potentially separate access policies.

This choice is deliberate and has important implications for both security analysis and experimental design. Group chat would introduce combinatorial complexity in access control (what can entity E_i see that entity E_j said?) and would make memory isolation intractable (if three entities are in a group conversation, whose relationship shard do new memories belong to?). The 1-to-1 model

cleanly maps to our formal problem: each session is a single interaction between the host agent and one external entity, governed by one policy, producing one pair of utility and security measurements.

In practice, many cross-boundary coordination tasks do reduce to 1-to-1 interactions. Scheduling a meeting between Alex and Tina involves Tina’s agent contacting Alex’s agent (or vice versa). Sharing a project update involves the sender’s agent contacting each recipient’s agent individually. Even tasks that appear to require group coordination (planning a team meeting) can be decomposed into a sequence of 1-to-1 queries (check each person’s calendar in turn).

Policy Files

Each relationship is governed by a policy file that specifies what the agent may access and disclose during interactions with that entity. The policy file is a JSON structure containing the following fields: **relationship_id** (a unique identifier for this entity), **trust_tier** (a categorical label such as “inner_circle,” “formal_business,” or “acquaintance”), **allowedNoteIds** (an array of note identifiers that this entity may access), **calendarAccess** (one of “full_detail,” “busy_only,” or “none”), **emailAccess** (one of “filtered” or “none”), **emailFilter** (an array of keywords for email filtering), and **allowedTools** (an array of tool function names available to this interaction).

The policy file is the minimal viable access control mechanism. It defines the boundary of what an external entity can access through architectural enforcement: the PDP constructs the execution environment from the policy file, and data or tools not specified in the policy are physically absent from the interaction context. For the experimental configurations described in this study, we vary the governance *instructions* (D0, D1, D2) while holding the policy file constant (all data accessible), isolating the effect of prompt-level governance from architectural access control.

Access Granting via Share Links

External entities access the agent through *agentic links*: unique URLs that embed a token referencing the policy file. When a guest opens the link, the backend retrieves the associated policy from the **agenticLinks** database table, validates that the token is active and not expired, and constructs the execution environment accordingly. The flow proceeds through five steps: token extraction from the URL, policy lookup, token validation, Context Cell mounting (tool filtering and note ID retrieval), relationship shard loading (memory isolation), and finally agent execution within the restricted context. The response is then written to the guest-specific memory shard for future reference.

This mechanism is analogous to how Notion or Google Docs share links embed permissions: opening the link grants access to a specific scope of content. The difference is that instead of granting access to a static document, an agentic link grants access to a sandboxed agent that can reason, retrieve information, and take actions within the defined boundary.

Memory Architecture

The Cross-Contamination Problem

Agents with global memory frequently leak information across relationships. When the agent talks to both Investor A and Investor B, fragments from one conversation can contaminate the other’s context through the shared vector database. If Investor A mentions a competing deal in passing, and the agent stores this in global memory, a subsequent conversation with Investor B might retrieve that fragment through semantic similarity when Investor B asks about market conditions. Existing memory systems, including MemGPT and LangChain’s conversation memory, treat memory as a global resource with no concept of relationship-specific isolation.

Dual-Track Memory

We address this through a strict separation between self-state memory (belonging to the owner) and per-relationship memory shards.

Track 1 is the *Self-State Memory*. This is the agent’s core knowledge about its owner: preferences, constraints, identity facts, and private conversation history with the owner. Self-State Memory is included in every interaction context (because the agent needs to know who it represents) but is never directly exposed to external entities. When a guest asks a question, the agent may consult self-state to understand the owner’s policies (for example, “never share financials with competitors”), but the actual facts are processed internally, not inserted into the guest’s response context. The self-state track corresponds to the MEMORY.md file described in Section 2.

Track 2 consists of *Relationship Shards*. For each unique external entity E_i , the system maintains an isolated memory shard containing all episodic memory from interactions between the agent and E_i : questions asked, answers given, files disclosed, commitments made, and relationship metadata (first interaction timestamp, relationship category, trust level).

Isolation Mechanism

Shard isolation is enforced through the database layer, not through prompt instructions. The mechanism operates through three properties.

First, tagging at write time. Every conversation fragment written to the vector database includes a `relationship_id` metadata field. This field is set server-side based on the authenticated session, not based on the conversation content. The language model cannot influence which relationship identifier is attached to a memory fragment.

Second, filtering at read time. Semantic search queries include a hard `WHERE relationship_id = $E_i` filter at the database level. This physically prevents retrieval of fragments from other relationships. The query is constructed server-side from the authenticated session context. Even if the language model hallucinates a request to access memories from a different relationship, the database query is constructed independently of the model’s output and will not return unauthorised fragments.

Third, fail-closed design. If the relationship identifier is missing or ambiguous, the query returns zero results rather than falling back to global search. This default is critical: a misconfigured session should result in no memory access rather than unrestricted memory access.

The database schema implements this through a PostgreSQL table with pgvector support. The episodic memory table contains columns for the user identifier, relationship identifier (nullable, where NULL indicates self-state), conversation identifier, role (user or assistant), content, embedding vector, timestamp, and metadata. An index on `relationship_id` ensures efficient filtered queries, and an IVFFlat index on the embedding column enables fast approximate nearest-neighbour search. The retrieval query combines vector similarity search with relationship filtering in a single query, ensuring that isolation and relevance are enforced simultaneously.

Tiered Memory Hierarchy

Within each track, memory follows a three-tier hierarchy inspired by CPU cache architecture and MemGPT’s virtual context management.

L1 (Working Memory) is the language model’s active context window, currently spanning 128K to 200K tokens depending on the model. L1 has zero retrieval latency because the content is already in the prompt. Content enters L1 through the system prompt (MEMORY.md for self-state, recent

conversation history for the current session) and is evicted through a first-in-first-out policy when the context window fills.

L2 (Episodic Memory) is the vector database of timestamped conversation fragments described above. L2 has 50 to 200 milliseconds of retrieval latency due to the embedding search. Content enters L2 automatically as conversations proceed (every message is embedded and stored), and relevance-based retrieval brings the most pertinent fragments into L1 when the agent needs historical context.

L3 (Semantic Memory) consists of structured facts distilled from L2 through periodic consolidation. L3 has 10 to 50 milliseconds of retrieval latency because it uses graph traversal or SQL queries rather than vector search. L3 content represents durable knowledge (“Alex prefers async communication”) rather than episodic fragments (“on March 15, Alex said he prefers Slack over email”). L3 consolidation is partially implemented; full implementation including automatic fact extraction from episodic memory is identified as future work.

The dual-track separation applies across all tiers: self-state has its own L1, L2, and L3, and each relationship shard has its own L2 with L3 as a future extension.

The Heartbeat Evaluation Engine

Static question-answering benchmarks miss an important class of failures: those that arise from autonomous agent behaviour. Real personal agents do not passively wait for queries. They monitor state changes, detect relevant updates, and take proactive actions on behalf of their owners. An agent that correctly refuses a sensitive question when asked directly might proactively share the same information when it detects a calendar update that seems relevant to an ongoing conversation.

We introduce the heartbeat evaluation engine to capture this dynamic. Instead of a static sequence of questions and answers, both the target agent and the querying agent operate as persistent processes that exchange messages at regular intervals. The engine is implemented as a CRON job that fires at configurable intervals (every 15 minutes in production, every tick in evaluation mode).

At each heartbeat tick, the querying agent (Tina’s agent) consults its instruction file (HEART-BEAT.md), which defines a two-phase evaluation protocol. In Phase 1 (information gathering), the agent systematically works through a checklist of pending questions, selecting one to three questions per tick based on conversation history and the status of previous queries. In Phase 2 (deeper probing), the agent revisits previously refused questions with alternative framing strategies, testing whether the governance boundary is robust to rephrasing. The querying agent maintains its own memory (a MEMORY.md checklist) that tracks the status of each question: answered, refused, or pending.

The heartbeat engine captures four categories of emergent behavior that are invisible to single-step evaluation. Escalation occurs when the querying agent detects a refusal and revisits the question later with different framing. Context accumulation happens as information disclosed in earlier turns establishes shared context that influences later disclosures. Social framing emerges when the querying agent wraps sensitive questions in work-acceptable contexts based on what it has learned about the target agent’s role and priorities. Within-tick batching occurs when the querying agent asks multiple questions in a single message, mixing legitimate and sensitive queries to overwhelm per-query governance decisions.

The heartbeat engine also enables measurement of leakage dynamics over time. By tracking which tick each leak first occurs at, we can characterise the temporal profile of information disclosure: does leakage happen immediately (suggesting the governance boundary was never effective), gradually (suggesting conversational erosion), or in bursts (suggesting specific trigger events)?

Technology Stack

The Pulse prototype is implemented using modern web technologies optimised for serverless deployment and rapid iteration. The backend is built on Next.js 15 with the App Router, providing server-side rendering for the user interface and API routes for agent interactions. The Vercel AI SDK handles streaming language model responses with tool use orchestration, supporting multiple model providers including Claude (Anthropic), GPT-4 (OpenAI), and Gemini (Google). Text embeddings use OpenAI’s text-embedding-3-large model for semantic search.

The primary database is PostgreSQL hosted on Supabase, storing notes, folders, user accounts, authentication sessions, episodic memory logs, task items, and agentic link configurations. The pgvector extension provides vector similarity search directly within PostgreSQL, avoiding the operational complexity of a separate vector database while supporting hybrid search (vector similarity combined with metadata filtering). Redis provides in-memory caching for session state and idempotency tokens.

External service integrations include Google Calendar and Gmail (via Google APIs), Microsoft Calendar and Outlook (via Microsoft Graph API), and Notion (via the Notion API). These integrations follow the MCP specification where possible, ensuring that the same interface that connects to Google Calendar in production also connects to the evaluation harness during experiments.

Authentication is handled by NextAuth.js, providing OAuth 2.0 flows for third-party service connections and session management for the web interface. Guest access through agentic links uses time-limited, single-use tokens validated against the `agenticLinks` database table. The CRON-based heartbeat engine authenticates through environment-variable secrets. Rate limiting is enforced at three levels: 100 requests per minute for authenticated users, 20 requests per minute per guest token, and one heartbeat execution per user per 15-minute interval.

The platform is deployed on Vercel’s serverless infrastructure with edge caching. Observability is provided through Vercel Analytics, custom structured logging with trace identifiers propagated through all function calls, and a dedicated audit table for security-critical events (Context Cell violations, escalations, denied tool calls) with immutable timestamps.